Assignment-1

Foundations & Algorithmic Analysis

EXPLORING AND IMPLEMENTING DIFFRENRNT PARADIGMS

Institute Name- K.R. Mangalam University

Course Name: Data Structures

Program Name: B. Tech Computer Science and Engineering (AI and ML)

Faculty Name: Dr. Neetu Chauhan

BUILDING A PYTHON AERT TO LEARN DIFFERENT PARADIGMS

By: Mehul Srivastava

Roll No.- 2501730149

1. Introduction:

This assignment implements a small toolkit called Algorithmic Efficiency and Recursion Toolkit (AERT) using Python. Here, we aim to understand basic data structures, abstract data types. recursion, and algorithm efficiency. The toolkit includes:

1. Stack ADT implementation
2. Recursive factorial Fibonacci (naive and memorized versions)
3. Tower of Hanoi
4. Recursive binary search

2. Stack ADT:

2.1. What is ADT?

An Abstract Data Type (ADT) is a logical description of a data structure that defines the operations that can be performed on it without specifying how those operations are implemented internally.

For example, a Stack ADT allows operations like:

- push
- pop
- peek
- is_empty
- size

The user only knows what the operations do, not how they are implemented.

2.2. Why stack operations are usually O (1) (constant time)?

Most stack operations take constant time O (1) because they operate only on the top element of the stack.

For example:

- push()- adds an element to the top
- pop()- removes the top element
- size()- check the length of the stack

Since no traversal of the structure is required, the time complexity remains constant.

3. Factorial & Fibonacci (Recursion + Complexity):

3.1.1. Time Complexity of Factorial:

Each recursive call reduces the value of n by 1, so the function runs n times.

$T(n) = T(n-1) + O(1)$

3.1.2. Space Complexity of Factorial:

Because recursion uses the call stack, the maximum depth is n

Hence the space complexity is O(n)

3.2.1. Time Complexity of Naïve Fibonacci:

The naive version directly follows the recursive definition. However, it recomputes many values multiple times.

Hence the Time Complexity for Naïve Fibonacci is $O(2^n)$

3.2.2. Time Complexity of Memorized Fibonacci:

Memorization stores previously computed results in a dictionary. When the function needs a value, it first checks whether it has already been calculated. If yes, the stored value is returned immediately.

Hence the Time Complexity of Memorized Fibonacci is O (n)

3.2.3. Space Complexity of both Naïve and Memorized Fibonacci:

Since both have the same recursion depth, the space complexity for both of them is same, i.e. – O (n).

3.2.4. Why naive Fibonacci becomes slow for large n:

Since the Time Complexity for the naïve Fibonacci is $O(2^n)$, it increases exponentially as the value of n increases, so for large n, the time complexity increases exponentially and it slows down significantly, for eg:

- For n = 2, time complexity is $2^2 = 4$
- For n = 5, time complexity is $2^5 = 32$
- For n = 10, time complexity = $2^{10} = 1024$

4. Tower of Hanoi:

4.1.1. Why number of moves becomes 2n −1?

To move n disks from the source rod to the destination rod using an auxiliary rod, the process is:

- Move n−1 disks from source to auxiliary
- Move the largest disk from source to destination
- Move n−1 from auxiliary to destination

Which comes to be:

$$\boxed{T(n) = T(n-1) + 1 + T(n-1)}$$

Which simplifies to after ignoring constants:

$$\boxed{T(n) = 2(n-1) + 1}$$

4.1.2. Time Complexity:

Time Complexity for Tower of Hanoi is O (2^n)

4.1.3. Space Complexity:

Space Complexity for Tower of Hanoi is O (n)

5. Recursive Binary Search (Divide & Conquer + Recurrence)

5.1.1. Why Time Complexity of Binary Search is O (log(n))

Binary search reduces the search space by half at every step. Therefore the recurrence relation is T(n) = T(n/2) + O(1), where T(n/2) represents searching in half of the array and O(1) represents the constant time needed for comparison. Solving this recurrence gives a time complexity of O(log (n)).

5.2.1. Best Case Scenario for Binary Search:

Best case will be when the element will be found exactly in the middle of the array, giving us the time complexity of O(1), meaning that no traversal will be required and no further division.

5.2.2. Average Case Scenario for Binary Search:

In the average case, the algorithm needs to check several middle elements before finding the target. Since the search space is reduced by half at each step, giving us the time complexity of O(log (n)).

5.2.3. Worst Case Scenario for Binary Search:

The worst case occurs when the element is not present, or the element is located at the last possible position examined. The algorithm keeps dividing the array until only one element remains, giving us the time complexity of O(log (n)).

6. Short Paradigm Reflection:

6.1.1. Why Binary Search is divide and conquer?

Binary search follows the Divide and Conquer approach because the problem is repeatedly divided into smaller parts.

First, the algorithm checks the middle element of the sorted array. If the target element is smaller, it searches only the left half. If it is larger, it searches only the right half. In each step, the search space is reduced to half of its previous size. Because the problem is divided into smaller subproblems and solved recursively, binary search is an example of the Divide and Conquer paradigm.

6.2.1. Why is Fibonacci Memorization an example of dynamic programme and what problem does it solve?

Memorized Fibonacci is an example of Dynamic Programming because it stores previously calculated results and reuses them.

In the naive recursive Fibonacci algorithm, the same Fibonacci values are calculated multiple times. For example, F(5) requires computing F(3) and F(4) and F(4) again requires F(3). This leads to many repeated calculations.

Memorization solves this problem by storing the results in a dictionary (or table). When the algorithm needs a value that has already been computed, it simply retrieves it instead of calculating it again. This reduces the number of recursive calls and improves efficiency.

6.3.1. Idea and working of Tower of Hanoi:

The Tower of Hanoi problem is solved using recursion because the problem can be broken into smaller versions of itself.

To move n disks from the source rod to the destination rod, the algorithm first moves n−1 disks to the auxiliary rod, then moves the largest disk, and finally moves the n−1 disks from the auxiliary rod to the destination rod.

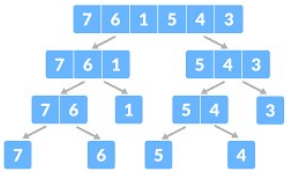
The base case occurs when there is only one disk. In that case, the disk can be moved directly from the source rod to the destination rod.

Hence this is the recursive solution towards the problem of tower of Hanoi

6.2. Real life examples of algorithm paradigms:

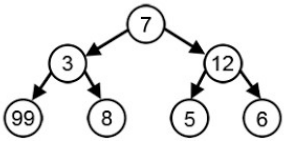
6.2.1. Divide and Conquer:

Searching for a word in a dictionary is similar to divide and conquer. Instead of checking every page, we open the dictionary near the middle and decide whether to search the first half or the second half.



6.2.2. Dynamic Programming:

Navigation systems such as Google Maps use dynamic programming concepts when calculating routes. They store intermediate results so that the same path calculations do not have to be repeated.

6.2.3. Greedy Approach:

A greedy approach can be seen when selecting the shortest path at each step while travelling. At every decision point, the option that appears best at that moment is chosen without considering all future possibilities.